

# Lagrangian Weighting for Data-Density-Aware Offline Reinforcement Learning

Theresa Geber, Maram Hadhri, Jonas Lang, Gerrit Grätz

*Applied Reinforcement Learning*

*Ludwig Maximilian University*

Munich, Germany

**Abstract**—Model-based offline reinforcement learning policies can exploit regions of state-action space where the learned transition model is unreliable due to sparse training data. A common mitigation penalizes the reward by a data-density cost signal, scaled by a fixed weight that must be hand-tuned per environment. We instead treat the density constraint as a constrained MDP and learn the penalty weight online via Lagrangian dual ascent, so the practitioner specifies an allowed constraint-violation rate instead of an arbitrary penalty magnitude. We compare this Lagrangian weighting scheme against the fixed-weight baseline on a pendulum swing-up task with an excluded low-data zone, and test generalization on LunarLander. *TODO: fill in with final headline result once experiments are complete.*

**Index Terms**—Reinforcement learning, Offline RL, Lagrangian optimization, Constrained MDP, Data density

## Contents

|                                               |   |
|-----------------------------------------------|---|
| I) Introduction .....                         | 3 |
| I.A) Contributions .....                      | 4 |
| II) Background / Problem Formulation .....    | 4 |
| III) Methodology .....                        | 5 |
| III.A) Environments .....                     | 5 |
| III.B) Methods compared .....                 | 5 |
| III.C) Training and evaluation protocol ..... | 5 |
| III.D) Metrics .....                          | 6 |
| IV) Results .....                             | 6 |
| V) Discussion .....                           | 6 |

|                      |   |
|----------------------|---|
| VI) Conclusion ..... | 7 |
| References .....     | 8 |

## I. Introduction

Model-based reinforcement learning agents that are trained or evaluated offline rely on a learned transition model to stand in for the real environment. That model is only accurate where it has seen enough data: outside those regions its predictions are unreliable, and a policy optimized against it can learn to exploit exactly the states where the model is wrong, producing behavior that looks good under the learned dynamics but fails, or is unsafe, under the true dynamics. This is especially problematic when parts of state space are inherently under-sampled in the offline dataset – not due to a data collection oversight, but because those regions are rare, costly, or dangerous to visit in the first place.

A standard mitigation is to penalize the reward with a data-density cost signal: states or state-action pairs that are far from the training distribution incur an additional cost, discouraging the policy from visiting them. (*TODO: cite the reference paper, ES2025-194, once entered in refs.bib*) scales this penalty by a fixed coefficient  $p$ , chosen by hand and held constant for the duration of training. This works, but the coefficient carries all of the burden: too small, and the policy still routes through the excluded, low-data region; too large, and the penalty dominates the task objective and cripples performance elsewhere. Because the “right” value of  $p$  depends on the environment, the task reward’s scale, and the shape of the excluded region, it must be re-tuned by hand whenever any of those change, and it cannot adapt over the course of training as the policy’s visitation distribution shifts.

We instead treat staying out of the low-data region as a constraint rather than as an unconstrained penalty term, and cast the problem as a constrained Markov decision process. The penalty weight becomes a Lagrange multiplier, updated online via dual ascent so that it grows when the policy violates the constraint more often than allowed and shrinks otherwise. Under this formulation, the practitioner no longer chooses a penalty magnitude directly; instead they specify an **allowed violation rate**  $\epsilon$  – a quantity with a direct operational meaning – and the weight is learned to enforce it. We compare

this Lagrangian weighting scheme against the fixed-weight baseline under otherwise identical conditions: the same data-density cost signal, the same agent, and the same evaluation protocol, so that any difference in outcome is attributable to the weighting scheme alone.

#### A. Contributions

- A shared, pluggable reward-wrapper implementation in which the baseline and Lagrangian conditions differ **only** in the weight component – same cost signal, same agent, same evaluation protocol – so the comparison isolates a single design choice.
- An empirical comparison of fixed-weight and Lagrangian-weight penalization on a pendulum swing-up task with an excluded, low-data zone, including how each method’s performance and safety trade-off responds to its own hyperparameter sweep.
- A generalization test on LunarLander, checking that any advantage observed on the pendulum task is not an artifact of its particular geometry or excluded-zone construction.

## II. Background / Problem Formulation

- Penalized reward formulation:

$$\text{effective reward} = R(s, a) - w(c) \cdot c(s, a) \quad (1)$$

where  $R(s, a)$  is the clean task reward,  $c(s, a)$  is a data-density cost signal, and  $w(c)$  is the penalty weight.

- Cost signal used: KDE density estimate, held fixed across both conditions so the comparison isolates the weighting axis only.
- Two weighting schemes under test:
  - Fixed weight:  $w(c) = p$ , a constant (paper baseline).
  - Lagrangian weight:  $w(c) = \alpha$ , updated online via

$$\alpha \leftarrow \max(0, \alpha + \eta(C - \varepsilon)) \quad (2)$$

where  $C$  is the observed constraint value (violation rate or mean cost) and  $\varepsilon$  is the target.

- Why this isolates a clean research question: transition model, agent, evaluation loop, and seeds are identical between conditions – only **how much** to penalize differs.

### III. Methodology

#### A. Environments

- **Primary: Pendulum swing-up with excluded zone.** One-sided low-data band the agent must learn to avoid while still reaching upright (env E1); shifted/re-labeled variant (env E3) to test whether Lagrangian auto-tuning survives a change in the “correct”  $p$ .
- **Generalization: LunarLander.** *TODO: define the excluded-zone / low-data-region analogue for this environment, and why it is expected to stress-test the same claim.*
- Shared components held fixed across both methods per environment: transition model / simulator, SAC hyperparameters, training steps, seeds.

#### B. Methods compared

- baseline: KDE density cost + fixed weight, swept over  $p \in \{\dots\}$ .
- lagr: KDE density cost + Lagrangian weight, swept over  $\varepsilon \in \{\dots\}$ .
- Ensemble- and BNN-based uncertainty cost signals are out of scope for this paper.

#### C. Training and evaluation protocol

- Offline dataset generation per environment.
- SAC training,  $N$  steps,  $\geq 5$  seeds per cell, mean  $\pm$  std reported.
- Evaluation always on the clean, unpenalized reward, so methods with different training objectives remain comparable.
- Fixed start state, fixed episode length, greedy (deterministic) action evaluation.

#### D. Metrics

- **Safety (lower is better):** zone visit rate (episode-level), zone step fraction, left/right path split (route-around behavior).
- **Task performance (higher is better):** true return mean/std, upright success rate, time-to-upright.
- **Constraint behavior (Lagrangian only):** final  $\alpha$ ,  $\alpha$  trajectory over training, constraint value  $C$  vs.  $\varepsilon$ , constraint-satisfied flag.
- **Robustness / tunability:** spread of (return, zone rate) across each method’s own knob sweep; a flat spread means the method is easy to tune – the Lagrangian’s expected advantage.
- **Compute:** wall-clock training time, where relevant to a claim.
- For LunarLander: same categories, redefined for that environment’s task-success and “excluded zone” analogue.

#### IV. Results

- Pendulum (E1 / E3): decision table comparing baseline vs. lagr on return, zone rate, and robustness score.
- Training dynamics:  $\alpha$  trajectory plot; constraint  $C$  vs.  $\varepsilon$  over time.
- Sweep plots: return / zone-rate as a function of  $p$  (baseline) vs.  $\varepsilon$  (lagr) – the robustness comparison.
- Path analysis: left / right route histograms.
- LunarLander: same table and plots, framed as a generalization check – does the pendulum ranking hold outside that setting?

*TODO: insert tables and figures once the full run matrix is aggregated.*

#### V. Discussion

- Interpretation: why learning  $\alpha$  outperforms a fixed  $p$  – it adapts to how much penalty is actually needed to satisfy the constraint, rather than requiring a priori knowledge.

- Where the Lagrangian underperforms or adds complexity: extra hyperparameters ( $\eta$ ,  $\varepsilon$ ,  $\alpha_{\text{init}}$ ), sensitivity to update frequency, transient behavior early in training.
- What the LunarLander result adds or complicates: confirms generalization, or surfaces an environment-specific caveat.
- Limitations: single cost signal (KDE) tested; constraint definition (zone rate vs. mean cost) changes what  $\varepsilon$  means; seed count; compute budget.
- Future work: combining Lagrangian weighting with the uncertainty-based cost signals (ensemble / BNN), flagged as out of scope here but a natural extension.

## VI. Conclusion

Lagrangian dual-ascent weighting removes the need to hand-tune a fixed penalty coefficient and is at least as robust and performant as the fixed-weight baseline, on both the original pendulum task and a structurally different environment (LunarLander). *TODO: tighten once final results are in.*

## References